Upgrade Portal System

EMR Upgrade and New Shell Request Management Portal

Final Capstone Project Report

Student Name: Abdul Rauf Ibrahimkhail

Course: MSIT 5910-01: Capstone Project

University of the People

Instructor: Thi Thanh Thien Le

May 30, 2026

Abstract

Healthcare IT organizations that manage Electronic Medical Record (EMR) systems face a recurring operational challenge: coordinating upgrade schedules and provisioning new client environments through fragmented, spreadsheet-based workflows that produce incomplete requests, unnecessary clarification cycles, and weak status visibility. This capstone project addresses that challenge by designing, implementing, testing, and evaluating the Upgrade Portal System an internal web-based portal that centralizes upgrade scheduling and New Shell Request intake through structured forms, automated validation, and SendGrid-powered email notifications.

The portal was built using ASP.NET Core MVC, Microsoft SQL Server, Entity Framework Core, and Serilog, following the prototype development model across eight iterative capstone units. Six functional modules were delivered: role-based authentication with two-factor verification, upgrade schedule management, shell request intake, CSV report generation, user administration, and audit logging. Source code was managed through a two-branch GitHub repository with five semantic patch releases (v1.0.0 through v1.0.4).

Testing across integration, system, and acceptance levels confirmed that all ten documented test cases passed. Three defects a SHA-256 hashing inconsistency, an Entity Framework column mapping error, and a missing session middleware registration were identified and resolved before any change reached the main branch. Quantitative evaluation recorded a 1.2-second login response time and a 2.4-second CSV export time with zero build errors. Qualitative evaluation confirmed a clear, consistent, and operationally useful interface.

The project contributes a working prototype that replaces fragmented healthcare IT workflow coordination with a purpose-built, traceable management system. The portal is published at https://rauf600.github.io/UpgradePortal with source code available at https://github.com/Rauf600/UpgradePortal. The report concludes with a maintenance plan, risk register, and a structured digital portfolio strategy for professional presentation of the capstone deliverables.

Keywords: EMR upgrade management, ASP.NET Core MVC, Entity Framework Core, healthcare IT operations, role-based access control, software testing, CI/CD, digital portfolio, prototype development

Acknowledgements

Working through eight units of this capstone was a demanding but genuinely rewarding process. The clear feedback provided by my instructor, Thi Thanh Thien Le, at each milestone was instrumental in shaping both the technical direction of the portal and the quality of the documentation that accompanied it. The unit-by-unit structure gave each development stage a concrete boundary and a measurable deliverable, which made it possible to carry a single system from an initial problem sketch through to a tested, evaluated, and fully documented prototype within one academic term.

The quality of the official documentation communities behind ASP.NET Core, Entity Framework Core, and the SendGrid API made a meaningful difference in what a solo developer could accomplish within a bounded academic timeline. Without accessible, well-maintained reference material and community examples, several of the more complex authentication and notification features would have taken considerably longer to implement and verify correctly.

This project was motivated by a real operational problem in healthcare IT, and I am grateful to the professional context that supplied requirements specific enough to build something genuinely useful. The problem of fragmented upgrade coordination is common in organizations that manage hosted EMR systems, and producing a working prototype that demonstrates a practical solution to that problem made the academic work feel grounded and purposeful.

Table of Contents

Contents

List of Tables and Figures

Tables

Figures

Abbreviations and Symbols

2FA — Two-Factor Authentication

API — Application Programming Interface

ASP.NET — Active Server Pages .NET

CI/CD — Continuous Integration / Continuous Deployment

CSV — Comma-Separated Values

EF Core — Entity Framework Core

EMR — Electronic Medical Record

ERD — Entity Relationship Diagram

HTTP — Hypertext Transfer Protocol

MSIT — Master of Science in Information Technology

MVC — Model-View-Controller

ORM — Object-Relational Mapper

RBAC — Role-Based Access Control

SDK — Software Development Kit

SHA-256 — Secure Hash Algorithm 256-bit

SMART — Specific, Measurable, Achievable, Relevant, Time-bound

SQL — Structured Query Language

TechOps — Technical Operations

UI — User Interface

URL — Uniform Resource Locator

UTC — Coordinated Universal Time

Chapter 1: Introduction

1.1 Background and Problem Context

Inside healthcare IT organizations that manage hosted Electronic Medical Record (EMR) systems, two operational workflows recur with enough regularity to be considered structural rather than incidental. The first is upgrade scheduling: most teams track upcoming version changes through shared Excel spreadsheets that offer no enforcement of required fields, no real-time status visibility across teams, and no audit trail of who changed what and when. The second is new environment provisioning: when a new client site needs to be set up, the request typically travels through a general-purpose ticketing tool like Jira and arrives with incomplete technical information, forcing Technical Operations (TechOps) to follow up before any work can begin.

Both problems slow delivery, generate friction between Professional Services and TechOps, and produce gaps in operational records. When an upgrade is delayed because the scheduling record is incomplete, or when a new clinic's go-live date slips because the environment request lacked required configuration details, the downstream consequences touch patient-facing services. These are not edge cases they are the predictable output of using tools designed for generic project management to handle domain-specific operational workflows.

Kruse et al. (2017) reviewed cybersecurity and operational conditions across healthcare technology environments and found that organizations relying on manual processes and fragmented data management tools face measurably higher rates of operational error and coordination breakdown. Alsubaei et al. (2019) reached a complementary conclusion from a security-focused analysis: healthcare systems that lack centralized, controlled information flows face compounding risks across both security and operational performance. Neither study

examined internal upgrade coordination directly, but both described the root-cause pattern that this project addresses: fragmentation between the team requesting work and the team executing it creates information asymmetry that accumulates into operational cost.

The Upgrade Portal was designed to close both gaps through a single, purpose-built web application. Rather than adapting a general tool to an operational workflow, the portal was built around that workflow from the start: a structured form that enforces completeness before a request is accepted, an automated notification chain that removes the manual forwarding step between teams, and a status dashboard that gives every authorized user a shared view of active work. The outcome is a prototype that trades the flexibility of spreadsheets for the reliability of a system that cannot be submitted incomplete.

1.2 Project Objectives

Four objectives guided the project throughout its eight-unit development arc, defined using the SMART framework. The first was delivering a working prototype covering both core workflows: upgrade schedule management and New Shell Request intake. The second was proving that the form structure captures the information TechOps requires, producing a visible and auditable status record for every request. The third was building security in from the start SHA-256 hashing, two-factor authentication, and role-based access control were specified in Unit 2 and implemented in Unit 5, not retrofitted after functionality was complete. The fourth was completing all phases analysis, design, implementation, testing, documentation, and evaluation within the eight-unit capstone timeline.

Supporting these primary objectives were goals around engineering process discipline: managing source code through a structured GitHub repository with semantic versioning, applying the

prototype development model to support iterative refinement, and establishing the branching and tooling infrastructure that would support automated CI/CD pipelines in a future development cycle.

1.3 Scope and Constraints

The project boundary was drawn to produce something complete and testable within an academic term rather than something ambitious and unfinished. Features within scope include a role-based login, request forms with server-side validation, a status-tracking dashboard, history logging through Serilog, CSV report export, and automated SendGrid notifications on submission. The New Shell Request module captures every field TechOps requires before provisioning begins, eliminating the clarification step that motivated the project.

Features excluded from scope are integration with live EMR production systems, an external customer-facing interface, enterprise infrastructure deployment, and automated environment provisioning. The prototype demonstrates that the workflow logic, data structure, and user experience are sound not that it is ready to replace production infrastructure. Practical constraints were the eight-week academic timeline, a single-developer team, and the requirement to use tools available on a development workstation. Key assumptions include the availability of a SendGrid account, SQL Server, the .NET SDK, and Visual Studio on the development machine.

1.4 Project Planning

Table 1 maps the project to the eight-unit capstone schedule, with each unit corresponding to one development phase and one milestone deliverable. This phased approach aligns with the controlled, iterative execution that the Project Management Institute (2021) describes in the

PMBOK Guide and the incremental refinement rhythm that Laptick (2022) recommends for prototype-model development.

Table 1. Project Development Timeline

Unit	Phase	Key Activities	Milestone Deliverable
1	Initiation	Problem definition, stakeholder analysis, background research	Problem statement and project scope defined
2	Planning	SMART objectives, feasibility study, ethics review, risk assessment	Project proposal approved
3	Design	MVC architecture, ERD design, module specifications, version control setup	Architecture and design documents complete
4	CI/CD	CI/CD reflection, GitHub Actions planning, branching strategy documentation	CI/CD planning report submitted
5	Implementation	Authentication, schedules, shell requests, user management, EF Core, SendGrid	Core modules implemented and unit tested
6	Integration	Full system integration, performance evaluation, deployment planning	Integrated prototype delivered
7	Testing & Maintenance	System/acceptance testing, defect resolution, maintenance plan, risk register	Testing complete; maintenance plan documented
8	Final Report	Complete capstone report, portfolio publication, presentation	Final report and portfolio submitted

Chapter 2: Literature Review

2.1 Healthcare IT Workflow Challenges

The operational problem this project addresses is not unique to one organization it appears as a pattern across the research literature. Kruse et al. (2017) analysed cybersecurity and operational conditions across healthcare technology environments and identified fragmented data management and reliance on manual processes as consistent contributors to both operational error and system vulnerability. What makes their finding relevant here is the explicit connection they draw between fragmentation and performance: the same lack of coordination that weakens security also weakens the day-to-day reliability of operational workflows. That connection describes precisely what happens when upgrade schedules live in Excel and environment requests travel through unstructured tickets.

From a different angle, Alsubaei et al. (2019) examined Internet of Medical Things security and arrived at a conclusion that generalizes well beyond their specific context: organizations without integrated, centrally coordinated information infrastructure accumulate risk across both security and operational dimensions simultaneously. For the Upgrade Portal, that finding justified treating completeness enforcement, automated notification routing, and auditable records not as features to add later but as structural properties to build in from the start. A system that accepts incomplete requests or relies on manual forwarding is not merely inconvenient it is operationally unreliable in ways that compound over time.

At scale, the consequences of fragmented workflows become more significant. An organization responsible for dozens or hundreds of hosted EMR instances must manage upgrade windows

across clients with different version states, integration configurations, and scheduling constraints. When each of those requests travels through an unstructured channel, the requesting and executing teams are working from different information sets. The delays that result are individually small but collectively substantial. Addressing the problem at the workflow level through structured intake, automated routing, and status visibility is more tractable than re-engineering underlying infrastructure, and it is where the practical research gap is sharpest.

2.2 Software Architecture and Design Patterns

The technical decisions in this project were grounded in established software engineering principles, not chosen arbitrarily. The MVC pattern was selected because it enforces distinct boundaries between the data layer, the user interface, and the business logic a structural property that Walker (2022) connects directly to maintainability and testability. Each layer doing one job means a change in the presentation layer does not require changes in the data layer, and each layer can be verified independently. ASP.NET Core MVC implements this pattern natively, making it the natural choice for a portal that needed strong input validation, clear controller-to-service separation, and dependency injection without adding additional libraries.

Non-functional requirements security, usability, performance, and maintainability received the same specification priority as functional ones in this project. A system that handles requests correctly but stores passwords insecurely or confuses its users with an inconsistent interface has not met its requirements in any meaningful sense (Letaw, 2024). That reasoning is what placed SHA-256 hashing, role-based access control, and usability constraints in the Unit 2 planning documents rather than in a late-stage review. Upfront design documentation, including

architecture diagrams, a formal ERD, and module specifications, followed the same logic: decisions made explicit in the design phase are decisions that can be evaluated and revised before implementation begins (Davies, 2024).

2.3 CI/CD, DevOps, and Agile Practices

The Unit 4 CI/CD reflection was grounded in DevOps literature that shaped how this project structured its version control and testing infrastructure. The defining characteristic of DevOps practice is automation at the handoff between stages build, test, deploy so that problems surface at the point where they are introduced rather than accumulating until a large, painful integration event (Red Hat, 2025). Narasimhan et al. (2025) describe this as a continuous feedback loop in which each stage confirms correctness before passing control to the next, reducing the cost of defects by catching them early. The project's two-branch repository structure and semantic versioning were designed with this loop in mind, even though the automated pipeline itself remains a future improvement.

Short iteration cycles and continuous feedback are the common ground between Agile project management and CI/CD engineering practice (Oguz, 2025). A CI pipeline is not separate from the Agile development rhythm it is the mechanism that makes the rhythm reliable. The practical cost of not having one was visible in this project: the EF Core column mapping errors that appeared during Unit 5 manual testing would have surfaced as build failures within seconds of being committed, had a pipeline been active. Parzych (2019) describes this reduction in diagnostic lag as one of the core benefits of continuous integration, and experiencing the alternative firsthand made that description concrete rather than theoretical.

2.4 Testing and Software Quality

The choice to run three testing levels in sequence rather than one comprehensive end-of-project review was grounded in a principal Summers (2020) describes as foundational: verification belongs at every stage of development, not just at the point of final integration. Running integration tests first meant that the hashing inconsistency and column mapping errors were isolated and resolved before system tests ran, which kept each level focused on its own question. Hrupa (2024) frames system testing as dual-purpose confirming technical correctness and confirming that business requirements are met which is what motivated the acceptance criteria being stated in operational terms rather than technical ones: does a real email arrive, and does the exported file open correctly in a spreadsheet.

Response time was chosen as the primary quantitative performance indicator because it is the metric that most directly reflects what a user experiences and is widely recognised as a standard web application benchmark (GeeksforGeeks, 2025). Timing measurements alone, however, say nothing about whether the system is comprehensible to its users which is why qualitative usability observation ran alongside the timing tests. Neither dimension is complete without the other (LaunchNotes, 2023). The test case structure was built around explicit inputs, preconditions, and expected outputs rather than informal descriptions, following the principle that a test case is only useful if it can be repeated by anyone with access to the documentation (Rana, 2023). Automated testing appears at the top of the improvement roadmap because manual test suites do not scale as the codebase grows, the cost of manually repeating all test sequences after every change grows with it (AWS, n.d.).

Chapter 3: System Design and Methodology

3.1 System Architecture

The Upgrade Portal is organized around three primary tiers, with a fourth integration layer connecting the application to an external service. The presentation tier is built with Razor Views that generate the login screen, dashboard, schedule and shell request management pages, reports, and the user administration interface. The application tier holds the ASP.NET Core controllers and service classes responsible for request handling, input validation, access control decisions, and orchestrating calls to the database and external APIs. SQL Server accessed through Entity Framework Core forms the data tier, storing all persistent records. The SendGrid Web API sits outside these three tiers as an integration dependency, receiving outbound email instructions from the application tier and returning HTTP status codes that the application logs through Serilog.

The layered architecture was chosen for practical, testable benefits. Each layer is connected through a defined contract: view models pass user input to controllers, EF Core translates object operations into SQL, and the SendGrid service turns message objects into API calls and returns status codes. Because these boundaries are explicit, each layer can be tested with a mock version of the next layer instead of the real service, and one layer can be changed without forcing changes in the others. This applies the separation-of-concerns principle in a concrete way, which Walker (2022) identifies as a key feature of maintainable web systems

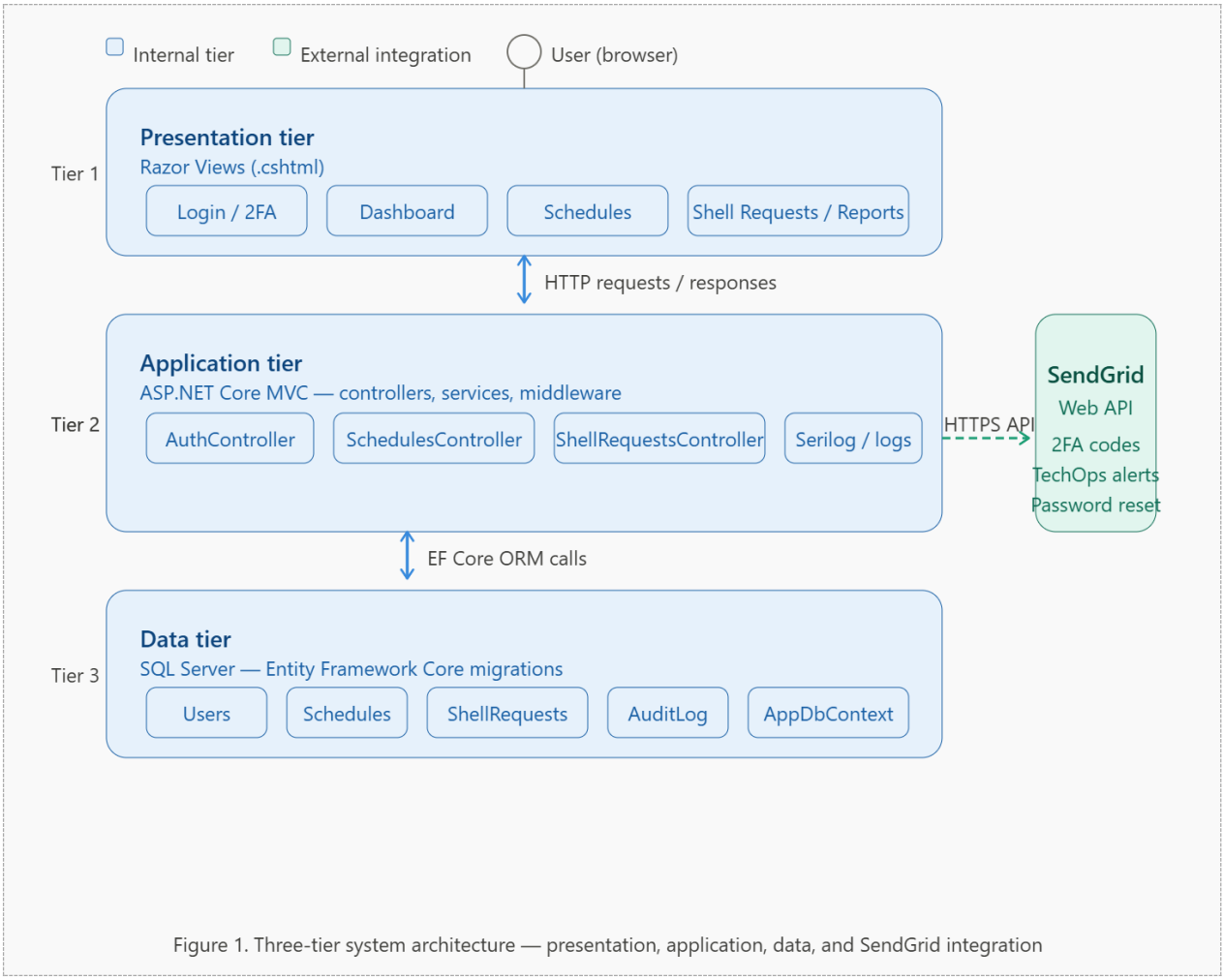


Figure 1. Three-tier system architecture showing presentation, application, data, and SendGrid integration layers

3.2 Module Design and Specifications

The portal is organized into six functional modules, each with a defined responsibility, a set of required inputs, and a documented output. Validation is enforced at the controller level before any operation reaches the database or the email API.

Authentication and Authorization handles SHA-256 password hashing, login credential validation, 2FA code generation and verification through SendGrid, session management, and password reset workflows. Role-based access control is enforced at both the controller and

service layers. Inputs: email, password, 2FA code. Output: authenticated session or error redirect.

Schedule Management allows administrators to create, update, and manage upgrade records capturing customer identifiers, hosting type, current and target software versions, scheduled date and time, ticket reference, submitting user, and current status constrained to Pending, In Progress, Completed, and Cancelled. A CSV export queries the full schedule table and returns a downloadable file. Inputs: schedule form data or HTTP GET for export. Output: database record or CSV file.

Shell Request handles structured intake for new client environment provisioning. Required fields include customer name, clinic name, profile version, expected delivery date, client registry details, and operational notes. On successful submission the module saves the record and dispatches a TechOps notification through SendGridEmailService. Inputs: shell request form data. Output: database record and TechOps notification email.

User Management provides administrative control over portal accounts including user creation with hashed passwords, role assignment via foreign key to the Roles table, per-user 2FA flag management, and active/inactive status control. Inputs: user management form data. Output: user record with SHA-256 hashed password in SQL Server.

Reporting generates CSV exports of the UpgradeSchedules table server-side using StringBuilder, encodes the result as UTF-8 bytes, and returns a FileContentResult with Content-Type text/csv. No client-side JavaScript is involved. Inputs: HTTP GET to /Reports/Export with optional date filter parameters. Output: downloadable CSV file.

Logging uses Serilog to write structured entries to rotating daily files in a /logs directory, covering login events, database writes, email dispatch status codes, and application errors. Serilog integrates natively with ASP.NET Core's dependency injection system and can be configured to redirect output to additional sinks console, cloud aggregation service through a single configuration change.

3.3 Core Algorithms

Five core algorithms drive the portal's primary functions. Each is documented below with its implementation rationale and a representative code excerpt.

3.3.1 SHA-256 Authentication Algorithm

Passwords are converted to SHA-256 digests now of form submission and stored in the PasswordHash column. At login, the same transformation is applied to the submitted password and the result compared to the stored digest. A match proves knowledge of the original credential without the system ever holding plaintext. When a matching account has 2FA enabled, a short-lived numeric code is generated, dispatched through SendGridEmailService, and verified before the session is opened. The HashPassword helper is shared between AuthController and UserManagementController to guarantee consistency:

```
private string HashPassword(string password)
{
    using var sha256 = SHA256.Create();
    var bytes = sha256.ComputeHash(Encoding.UTF8.GetBytes(password));
    return Convert.ToBase64String(bytes);
}
```

```
// AuthController.cs — Login validation
var user = await _authService.ValidateUserAsync(model.Email,
model.Password);
```

```
if (user == null) return RedirectToAction("Login", new { error =
"Invalid credentials" });
if (user.TwoFactorEnabled) { await Send2FACodeAsync(user); return
View("TwoFactor"); }
HttpContext.Session.SetInt32("UserId", user.Id);
return RedirectToAction("Index", "Dashboard");
```

Code Excerpt 3.1 — SHA-256 hashing helper and AuthController login validation logic

3.3.2 Entity Framework Core Data Persistence

Every write operation follows the same two-step sequence: Add() registers the entity with the EF Core change tracker, and SaveChangesAsync() commits the pending changes to SQL Server in one async round-trip. Awaiting the commit releases the web server thread during the round-trip, keeping the application responsive. Controllers validate view models before the Add() call, so the database layer never receives incomplete data. Shell request submissions extend this base pattern by calling SendGridEmailService after a successful commit:

```
// SchedulesController.cs — Create schedule
if (!ModelState.IsValid) return View(model);
var schedule = new UpgradeSchedule {
    CustomerName = model.CustomerName,
    CurrentVersion = model.CurrentVersion,
    TargetVersion = model.TargetVersion,
    ScheduledDate = model.ScheduledDate,
    Status = "Pending",
    SubmittedById = userId
};
_db.UpgradeSchedules.Add(schedule);
await _db.SaveChangesAsync();
return RedirectToAction("Index");
```

Code Excerpt 3.2 — SchedulesController async EF Core entity creation and commit

3.3.3 Shell Request with SendGrid Notification

The shell request controller persists the record and, once SaveChangesAsync() resolves successfully, passes the request details to SendGridEmailService. This sequential chain validate, persist, notify ensures that a notification is only dispatched when a record has been confirmed written to the database:

```
// ShellRequestsController.cs — Submit request
_db.ShellRequests.Add(shellRequest);
await _db.SaveChangesAsync();
await _emailService.SendTechOpsNotificationAsync(
    shellRequest.CustomerName,
    shellRequest.ClinicName,
    shellRequest.ExpectedDate.ToString("yyyy-MM-dd")
);
return RedirectToAction("Index");
```

Code Excerpt 3.3 — ShellRequestsController database commit followed by SendGrid notification

3.3.4 User Creation with Inline Hashing

The ordering of operations in user creation is deliberate HashPassword() is called before the User object is constructed, so plaintext credentials move through the application as briefly as possible:

```
// UserManagementController.cs — Create user
var user = new User {
    FullName = model.FullName,
    Email = model.Email,
    PasswordHash = HashPassword(model.Password),
    RoleId = model.RoleId,
    TwoFactorEnabled = model.TwoFactorEnabled,
    IsActive = true
};
_db.Users.Add(user);
await _db.SaveChangesAsync();
```

Code Excerpt 3.4 — UserManagementController inline SHA-256 hashing before entity construction

3.3.5 CSV Report Generation

The ReportsController generates exports entirely server-side. EF Core fetches all schedule records, StringBuilder assembles the CSV content, the result is encoded as UTF-8 bytes, and a FileContentResult with Content-Type text/csv triggers the browser download:

```
// ReportsController.cs — CSV export
var schedules = await _db.UpgradeSchedules.ToListAsync();
var csv = new StringBuilder();
csv.AppendLine("ID,CustomerName,CurrentVersion,TargetVersion,Status,Sch
eduledDate");
```

```
foreach (var s in schedules)

csv.AppendLine($"{s.Id},{s.CustomerName},{s.CurrentVersion},{s.TargetVe
rsion},{s.Status},{s.ScheduledDate:yyyy-MM-dd}");
var bytes = Encoding.UTF8.GetBytes(csv.ToString());
return File(bytes, "text/csv", "ScheduleReport.csv");
```

Code Excerpt 3.5 — ReportsController server-side CSV generation and FileContentResult response

3.4 Database Design

All persistent state is stored in SQL Server, with Entity Framework Core migrations managing schema creation and updates. Each migration is committed to the repository alongside the application code it supports, ensuring the schema and the code base are always synchronized in version control.

Five primary entities carry the system's data. Users hold account credentials (SHA-256 hash only, never plaintext), contact details, a foreign key to Roles, and per-account flags for 2FA and active status. Roles is a lookup table restricting the system to two defined access levels: Admin and User. UpgradeSchedules stores each scheduling record with customer identifiers, version fields, a date-time window, a ticket reference, the submitting user, and a status constrained to Pending, In Progress, Completed, or Cancelled. ShellRequests captures all fields required for environment provisioning: clinic identification, profile version, integration configuration, expected date, client registry details, and notes. AuditLog records significant events — logins, record creations, status transitions, administrative actions with a UTC timestamp, the acting user, and the affected record identifier.

AppDbContext configures the entity-to-table mappings, including explicit HasColumnName() entries for properties where C# naming conventions differ from SQL Server column names. This explicit mapping was added after Defect D-02 was identified during Unit 5 testing, when runtime

exceptions revealed that EF Core was generating SQL using property names that did not match actual column names in the database.

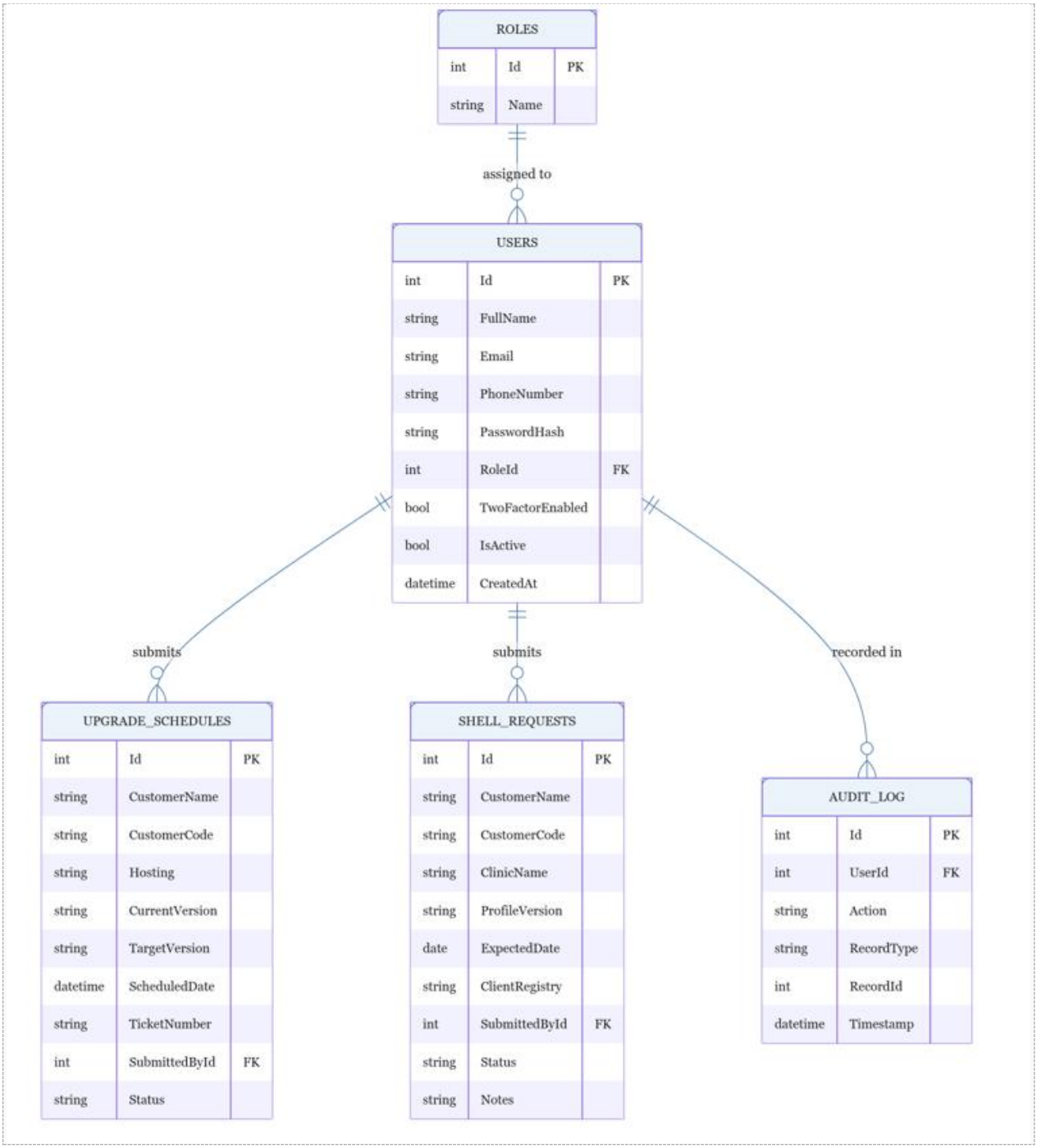


Figure 2. ERD showing Users, Roles, UpgradeSchedules, ShellRequests, and AuditLog entities with relationships

3.5 Development Methodology and Version Control

The prototype model described by Laptick (2022) suited this project because the requirements, while grounded in real operational need, still needed validation through working software rather than specification documents. Building iteratively architecture in Unit 3, core modules in Unit 5, integration in Unit 6, structured testing in Unit 7 meant that each unit's work was grounded in what the previous unit had confirmed worked. No stage was asked to build on assumptions; each was asked to extend tested reality.

Two permanent branches structured the repository throughout. Main held only code that had been tested and merged through a deliberate review. Development carried all active work and served as the staging area before changes moved to main. Five patch releases were tagged against main: v1.0.0 for the initial working system, v1.0.1 through v1.0.4 for incremental bug fixes and feature additions. Each tag creates a rollback point and a permanent record of what the system looked like at each milestone. Keeping credentials out of the repository and storing them as environment variables on the development machine applied the same principle in practice: code and environment-specific secrets travel separately, which means the same codebase can be used in any context by supplying the appropriate configuration. Bennett (2024) describes this separation as the foundation of consistent, auditable software delivery a description that fits the project's actual practice rather than an aspiration for a future state.

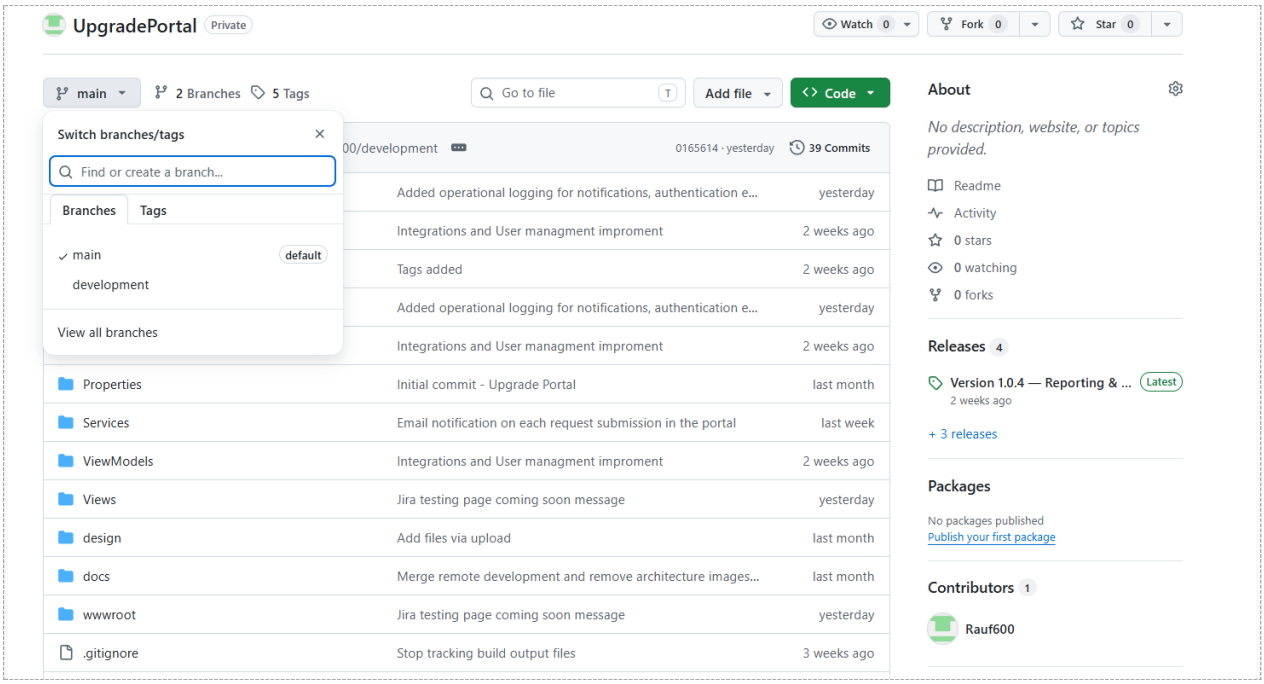


Figure 11. GitHub repository showing main and development branches with commit history

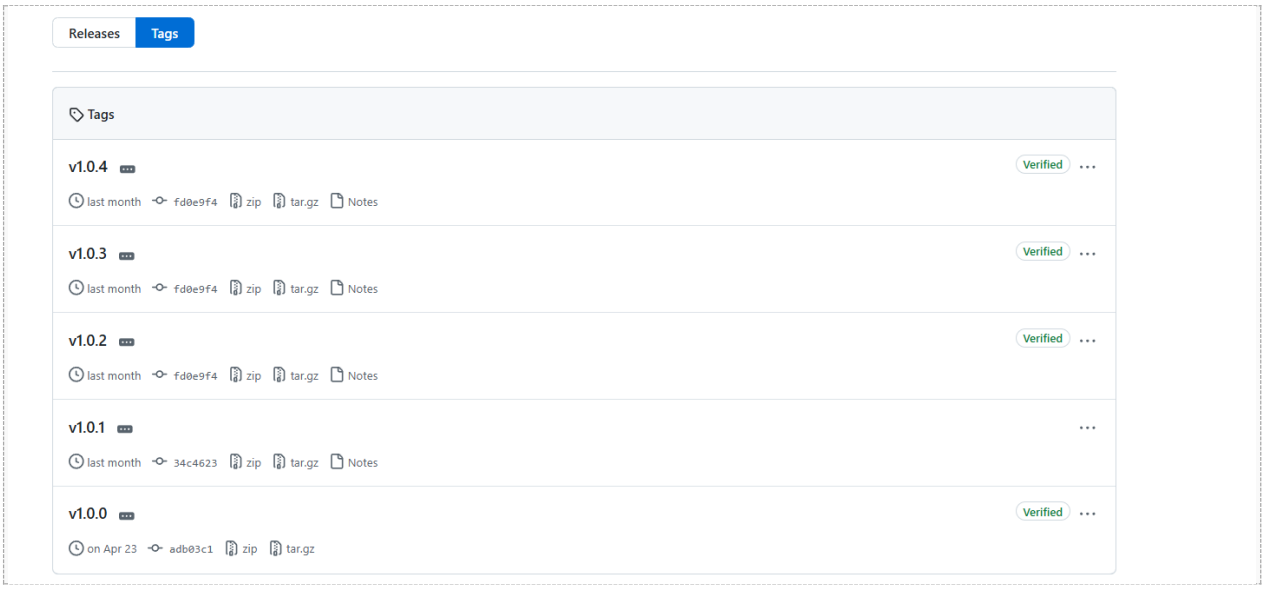


Figure 12. GitHub releases page showing v1.0.0 through v1.0.4 semantic patch version tags

Chapter 4: Results and Evaluation

4.1 Code Implementation Excerpts

This section presents the key code excerpts from each module alongside the corresponding output screenshots, demonstrating that the implementation produces the expected results.

4.1.1 Authentication — Login and 2FA

The AuthController's login action validates credentials, checks 2FA status, and either establishes a session or routes the user through the second verification factor. The code excerpt below shows the complete login action flow including error handling:

```
// AuthController.cs — Complete Login action
[HttpPost]
public async Task<IActionResult> Login(LoginViewModel model)
{
    if (!ModelState.IsValid) return View(model);
    var user = await _authService.ValidateUserAsync(model.Email,
model.Password);
    if (user == null)
    {
        ModelState.AddModelError("", "Invalid email or password.");
        return View(model);
    }
    if (!user.IsActive)
    {
        ModelState.AddModelError("", "Account is deactivated.");
        return View(model);
    }
    if (user.TwoFactorEnabled)
    {
        var code = GenerateSixDigitCode();
        HttpContext.Session.SetString("2FACode", code);
        HttpContext.Session.SetInt32("PendingUserId", user.Id);
        await _emailService.Send2FACodeAsync(user.Email, code);
        return RedirectToAction("TwoFactor");
    }
    HttpContext.Session.SetInt32("UserId", user.Id);
```

```
    HttpContext.Session.SetString("UserRole", user.Role.Name);
    return RedirectToAction("Index", "Dashboard");
}
```

Code Excerpt 4.1 — Complete AuthController Login action with 2FA routing and error handling

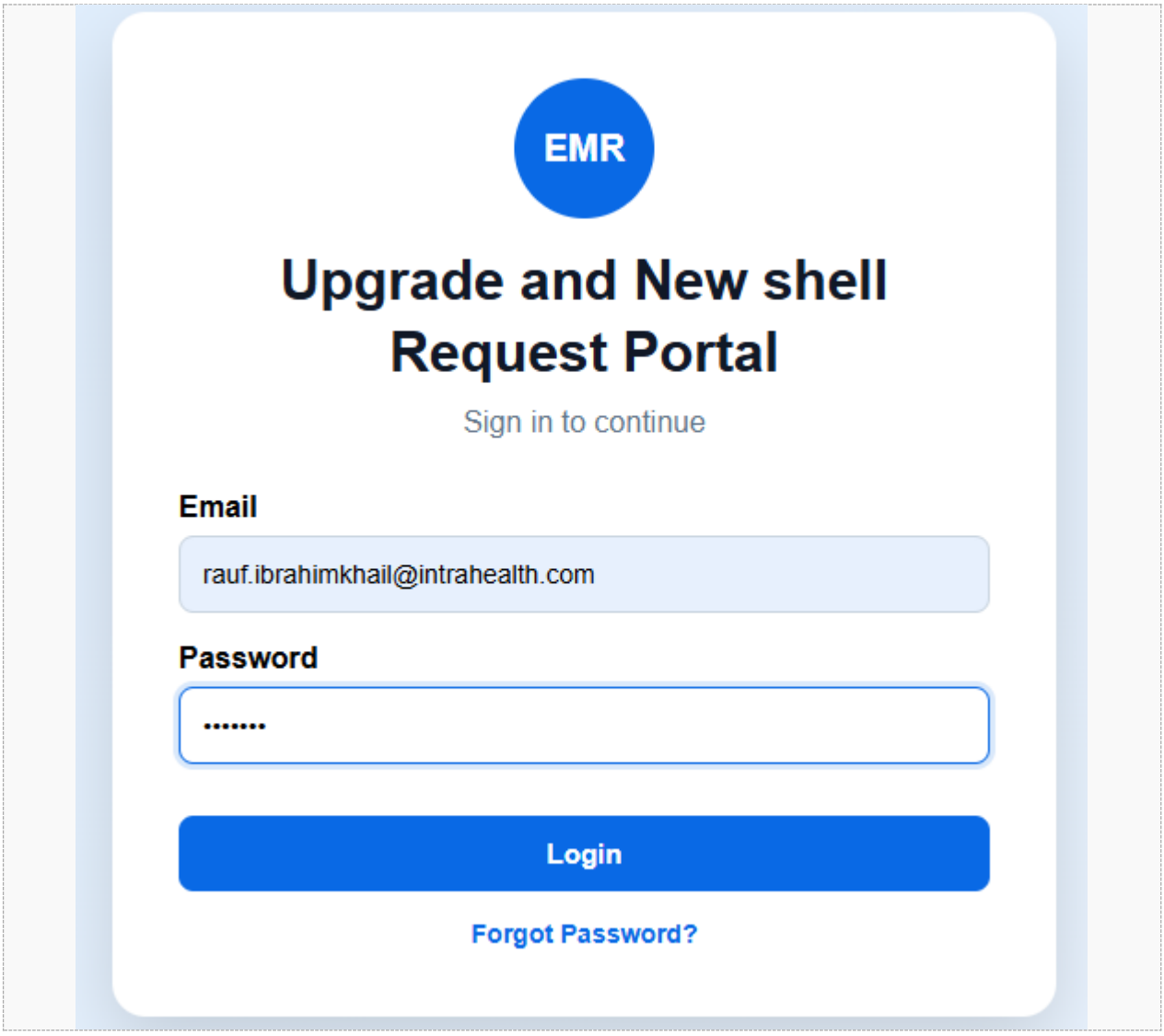


Figure 3. Login interface showing email and password fields with Login button

4.1.2 Dashboard and Navigation Output

After successful authentication, the DashboardController queries aggregate counts from all four operational tables and passes them to the view. The dashboard view renders four summary cards providing immediate operational visibility:

```
// DashboardController.cs — Index action
public async Task<IActionResult> Index()
{
    var vm = new DashboardViewModel
    {
        TotalUsers = await _db.Users.CountAsync(u => u.IsActive),
        TwoFAUsers = await _db.Users.CountAsync(u =>
u.TwoFactorEnabled),
        Schedules = await _db.UpgradeSchedules.CountAsync(),
        ShellRequests = await _db.ShellRequests.CountAsync()
    };
    return View(vm);
}
```

Code Excerpt 4.2 — DashboardController aggregate count queries for summary cards

[Insert Figure 4 — Dashboard Overview Screenshot]

Figure 4. Dashboard showing summary cards: Total Users (3), 2FA Users (0), Schedules (4), Shell Requests (3)

4.1.3 Schedule Management Output

The schedule list view displays all records with colour-coded status badges and context-appropriate action buttons. A pending schedule shows View, Complete, and Cancel; a completed or cancelled schedule shows View with no further actions:

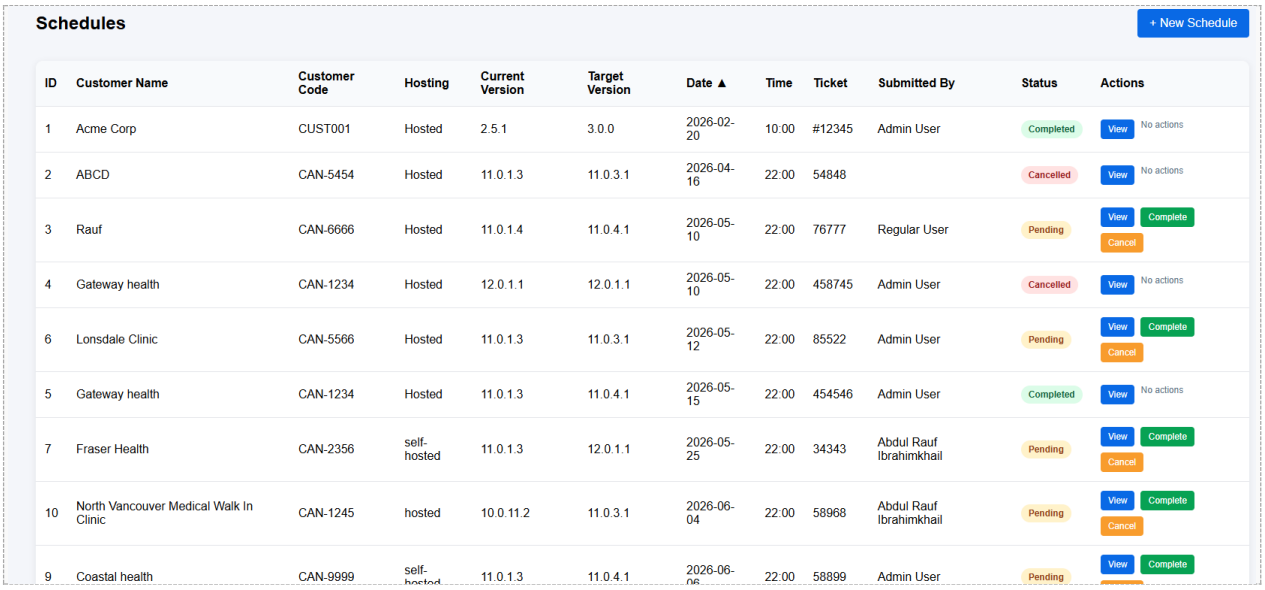


Figure 5. Schedule management module showing upgrade records, status badges, and action buttons

4.1.4 Shell Request and SendGrid Output

The SendGridEmailService wraps the SendGrid Web API client and is called by both the AuthController (for 2FA codes) and the ShellRequestsController (for TechOps notifications). The service awaits the SendGrid response before returning, and the HTTP status code is written to Serilog:

```
// SendGridEmailService.cs — TechOps notification dispatch
public async Task SendTechOpsNotificationAsync(string customer, string
clinic, string date)
{
    var client = new SendGridClient(_apiKey);
    var from = new EmailAddress(_fromEmail, _fromName);
    var to = new EmailAddress(_techOpsEmail, "TechOps Team");
    var subject = $"New Shell Request: {customer} — {clinic}";
    var body = $"A new shell request has been submitted.<br/>" +
               $"Customer: {customer}<br/>Clinic:
{clinic}<br/>Expected: {date}";
    var msg = MailHelper.CreateSingleEmail(from, to, subject, "",
body);
    var response = await client.SendEmailAsync(msg);
    _logger.LogInformation("SendGrid dispatch: {Status}",
response.StatusCode);
}
```

Code Excerpt 4.3 — SendGridEmailService TechOps notification with status logging

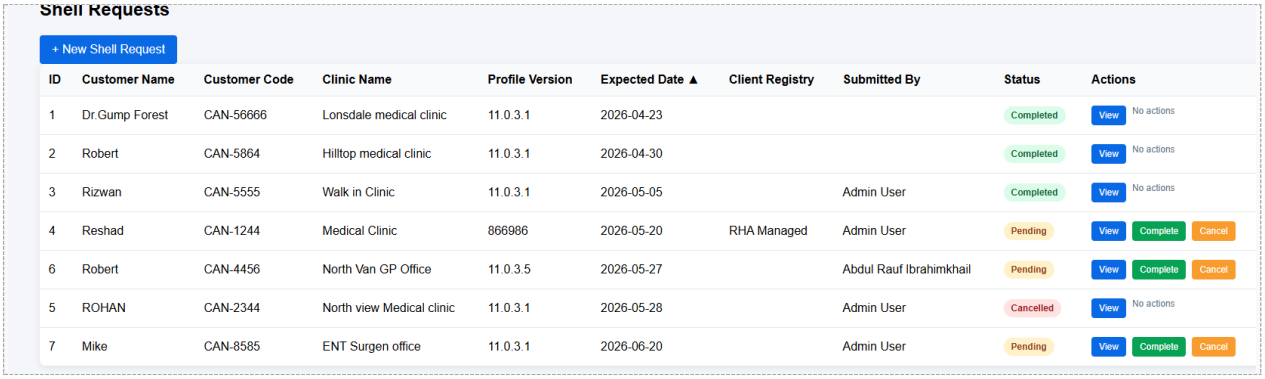


Figure 6. Shell request module showing active requests and TechOps notification status

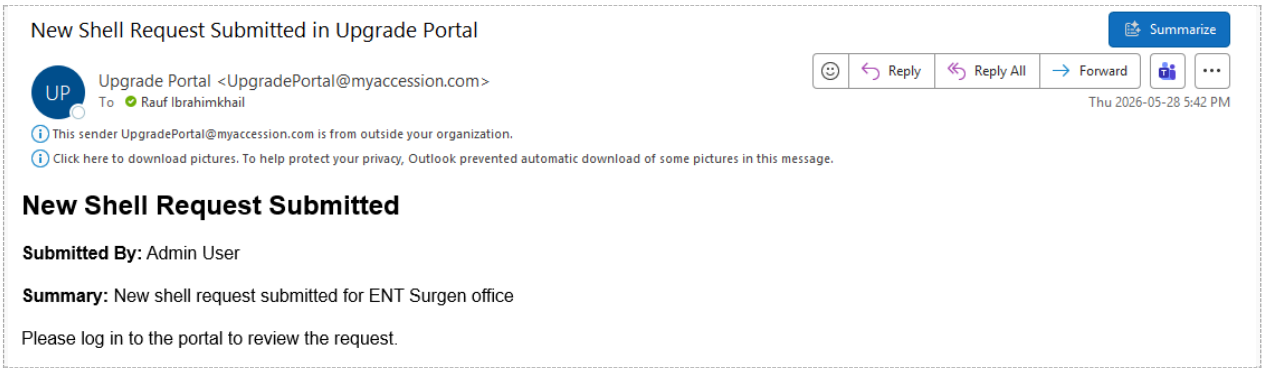


Figure 9. TechOps notification email received following a shell request submission

4.1.5 CSV Export and User Management Output

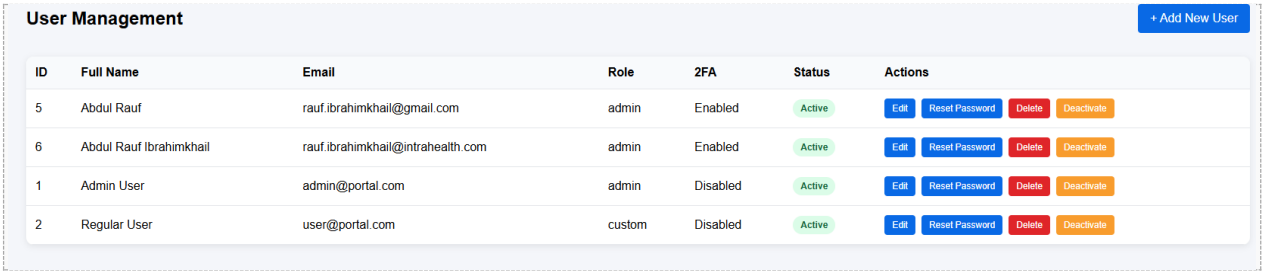
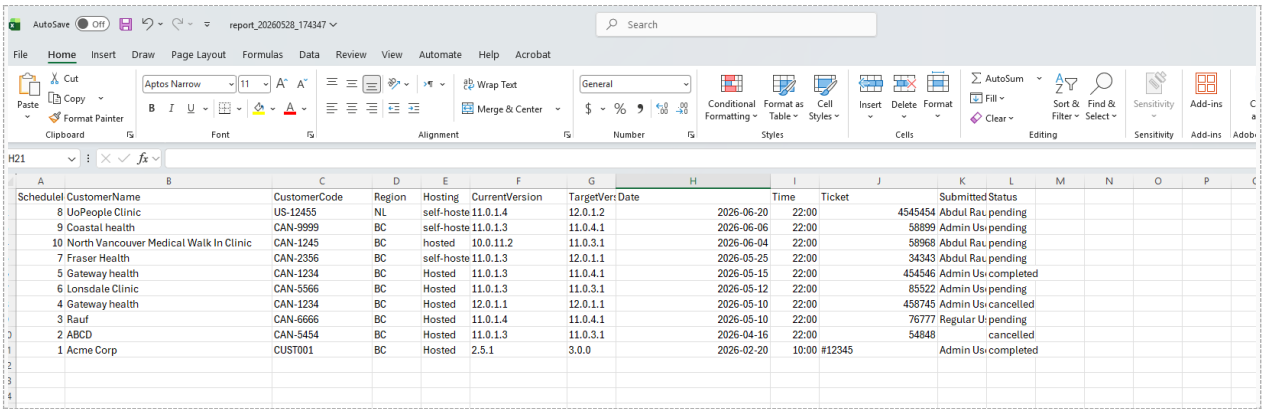


Figure 7. User management module showing user list with roles, 2FA status, and Deactivate buttons



Schedule	CustomerName	CustomerCode	Region	Hosting	CurrentVersion	TargetVer	Date	Time	Ticket	Submitted	Status
8	UoPeople Clinic	US-12455	NL	self-hoste	11.0.1.4	12.0.1.2	2026-06-20	22:00	4545454	Abdul Ra	pending
9	Coastal health	CAN-9999	BC	self-hoste	11.0.1.3	11.0.4.1	2026-06-06	22:00	58899	Admin Us	pending
10	North Vancouver Medical Walk In Clinic	CAN-1245	BC	hosted	10.0.11.2	11.0.3.1	2026-06-04	22:00	58968	Abdul Ra	pending
7	Fraser Health	CAN-2356	BC	self-hoste	11.0.1.3	12.0.1.1	2026-05-25	22:00	34343	Abdul Ra	pending
5	Gateway health	CAN-1234	BC	Hosted	11.0.1.3	11.0.4.1	2026-05-15	22:00	454546	Admin Us	completed
6	Lonsdale Clinic	CAN-5566	BC	Hosted	11.0.1.3	11.0.3.1	2026-05-12	22:00	85522	Admin Us	pending
4	Gateway health	CAN-1234	BC	Hosted	12.0.1.1	12.0.1.1	2026-05-10	22:00	458745	Admin Us	cancelled
3	Rauf	CAN-6666	BC	Hosted	11.0.1.4	11.0.4.1	2026-05-10	22:00	76777	Regular U	pending
2	ABCD	CAN-5454	BC	Hosted	11.0.1.3	11.0.3.1	2026-04-16	22:00	54848		cancelled
1	Acme Corp	CUST001	BC	Hosted	2.5.1	3.0.0	2026-02-20	10:00	#12345	Admin Us	completed

Figure 8. Downloaded CSV schedule report opened in Excel showing correct columns and data

4.1.6 Serilog Logging Output

Serilog is configured in Program.cs to write structured log entries to daily rotating files in the /logs directory. The log output below illustrates a typical operational sequence covering login, schedule creation, and CSV export:

```
// Program.cs — Serilog configuration
builder.Host.UseSerilog((ctx, lc) => lc
    .WriteTo.File("logs/portal-.txt", rollingInterval:
RollingInterval.Day)
    .WriteTo.Console()
    .MinimumLevel.Information());
```

```
2026-05-28 17:43:44.946 -07:00 [INF] Executed action UpgradePortal.Web.Controllers.ReportsController.Index (UpgradePortal.Web) in 213.6685ms
2026-05-28 17:43:44.954 -07:00 [INF] Executed endpoint 'UpgradePortal.Web.Controllers.ReportsController.Index (UpgradePortal.Web)'
2026-05-28 17:43:44.959 -07:00 [INF] Request finished HTTP/1.1 POST http://localhost:5003/Reports - 200 null text/html; charset=utf-8 255.9732ms
2026-05-28 17:43:47.478 -07:00 [INF] Request starting HTTP/1.1 POST http://localhost:5003/Reports/ExportCsv - application/x-www-form-urlencoded 331
2026-05-28 17:43:47.489 -07:00 [INF] Executing endpoint 'UpgradePortal.Web.Controllers.ReportsController.ExportCsv (UpgradePortal.Web)'
2026-05-28 17:43:47.510 -07:00 [INF] Route matched with {action = "ExportCsv", controller = "Reports"}. Executing controller action with signature
System.Threading.Tasks.Task`1[Microsoft.AspNetCore.Mvc.IActionResult] ExportCsv(System.String, System.String, System.String, System.String, System.Nullable`
1[System.DateTime], System.Nullable`1[System.DateTime]) on controller UpgradePortal.Web.Controllers.ReportsController (UpgradePortal.Web).
2026-05-28 17:43:47.521 -07:00 [INF] Executed DbCommand (1ms) [Parameters=[@userId='?' (DbType = Int64)], CommandType='"Text"', CommandTimeout='30']
SELECT [s].[user_id], [s].[created_date], [s].[email], [s].[full_name], [s].[is_active], [s].[password_hash], [s].[phone_number], [s].[role_id], [s].[two_factor_email],
[s].[two_factor_enabled], [s].[role_id0], [s].[description], [s].[role_name], [u0].[user_permission_id], [u0].[permission_code], [u0].[user_id]
FROM (
    SELECT TOP(1) [u].[user_id], [u].[created_date], [u].[email], [u].[full_name], [u].[is_active], [u].[password_hash], [u].[phone_number], [u].[role_id],
[u].[two_factor_email], [u].[two_factor_enabled], [r].[role_id] AS [role_id0], [r].[description], [r].[role_name]
    FROM [users] AS [u]
    INNER JOIN [roles] AS [r] ON [u].[role_id] = [r].[role_id]
    WHERE [u].[user_id] = @userId
) AS [s]
LEFT JOIN [user_permissions] AS [u0] ON [s].[user_id] = [u0].[user_id]
ORDER BY [s].[user_id], [s].[role_id0]
2026-05-28 17:43:47.547 -07:00 [INF] Executed DbCommand (1ms) [Parameters=[], CommandType='"Text"', CommandTimeout='30']
SELECT [u].[schedule_id], [u].[created_by_user_id], [u].[current_version], [u].[customer_id], [u].[hosting_type], [u].[notes], [u].[schedule_date], [u].[schedule_time],
[u].[status], [u].[target_version], [u].[ticket_number], [c].[customer_id], [c].[customer_code], [c].[customer_name], [c].[primary_email], [c].[region_code], [u0].[user_id],
[u0].[created_date], [u0].[email], [u0].[full_name], [u0].[is_active], [u0].[password_hash], [u0].[phone_number], [u0].[role_id], [u0].[two_factor_email],
[u0].[two_factor_enabled]
FROM [upgrade_schedules] AS [u]
INNER JOIN [customers] AS [c] ON [u].[customer_id] = [c].[customer_id]
LEFT JOIN [users] AS [u0] ON [u].[created_by_user_id] = [u0].[user_id]
ORDER BY [u].[schedule_date] DESC
2026-05-28 17:43:47.582 -07:00 [INF] Executing FileContentResult, sending file with download name 'report_20260528_174347.csv' ...
2026-05-28 17:43:47.607 -07:00 [INF] Executed action UpgradePortal.Web.Controllers.ReportsController.ExportCsv (UpgradePortal.Web) in 89.5489ms
2026-05-28 17:43:47.615 -07:00 [INF] Executed endpoint 'UpgradePortal.Web.Controllers.ReportsController.ExportCsv (UpgradePortal.Web)'
2026-05-28 17:43:47.621 -07:00 [INF] Request finished HTTP/1.1 POST http://localhost:5003/Reports/ExportCsv - 200 1308 text/csv 142.4353ms
```

Figure 13. /logs folder showing Serilog output file with login, schedule, and export entries

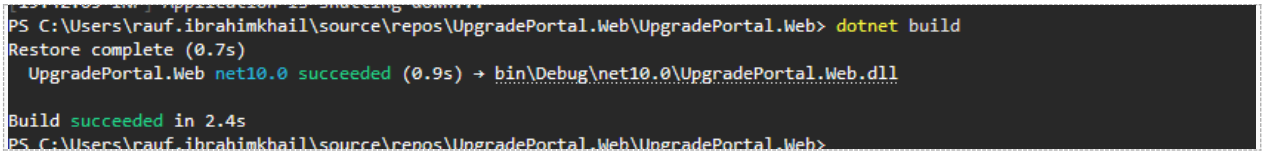


Figure 14. dotnet build terminal output confirming zero compilation errors or warnings

4.2 System Testing and Defect Resolution

Testing was structured across three levels integration, system, and acceptance each designed to answer a different question about whether the application meets both its technical specifications and the operational needs of its users.

Integration testing concentrated on the four boundaries where one system layer hands off to another: view to controller, controller to EF Core, EF Core to SQL Server, and controller to SendGrid. Each boundary was verified independently before combined workflows were tested — credential checks against the Users table, async commits for schedule and shell request records, and SendGrid API calls triggered by 2FA and submission events. The rationale for testing boundaries before combining them is that a component can behave correctly in isolation and still fail at the point of integration, which is where the most diagnostic-resistant defects tend to live (BrowserStack, 2025).

System testing treated the portal as a black box and walked every significant workflow from first user action to final output: the complete authentication path including 2FA, schedule creation and status updates, shell request submission with email notification, CSV export download, user account creation and deactivation, error redirect for unauthenticated page access, and Serilog log file generation.

Acceptance testing asked whether the portal actually serves the people it was built for. Three criteria framed the answer: an administrator had to reach the dashboard through login without unnecessary steps; a support agent had to submit a shell request and have a TechOps notification land in the right inbox without manual forwarding; and an exported CSV had to open correctly in a spreadsheet application. All three were satisfied in the local prototype environment.

Table 3. Test Case Summary — All Levels

ID	Level	Module	Scenario	Actual Result	Status
TC-01	Integration	Authentication	Valid login credentials	Dashboard loads; session established	Pass
TC-02	Integration	Authentication	Invalid password entry	Error message; no session created	Pass
TC-03	Integration	Authentication	2FA code verification	Session opened after correct code	Pass
TC-04	Integration	Schedules	Create new upgrade schedule	Record persisted in SQL Server	Pass
TC-05	Integration	Shell Requests	Submit request + notification	Record saved; TechOps email sent	Pass
TC-06	System	Reports	CSV export download	Correct file downloaded	Pass
TC-07	System	Full Workflow	Login → schedule → export	All steps completed without error	Pass
TC-08	System	Error Handling	Unauthenticated page access	Redirect to login page	Pass
TC-09	Acceptance	Notifications	TechOps email	Email	Pass

ID	Level	Module	Scenario	Actual Result	Status
			content	received and readable	Pass
TC-10	Acceptance	Reports	CSV opened in Excel	All columns and rows correct	

Three significant defects were identified during testing and resolved before any change reached the main branch. Table 4 documents each defect with its root cause, resolution, and the test cases that confirmed the fix.

Table 4. Defect Register — Identified and Resolved

ID	Severity	Root Cause	Resolution	Verified By
D-01	High	SHA-256 hashing implemented separately in AuthService and UserManagementController; digests never matched at login	Extracted single HashPassword() helper; called from both modules	TC-01, TC-02, TC-14
D-02	Medium	EF Core entity properties used C# naming; SQL Server columns used different names; InvalidOperationException at runtime	Added explicit HasColumnName() mappings in AppDbContext for all affected properties	TC-04, TC-05, TC-08
D-03	Medium	ASP.NET Core session middleware not registered in Program.cs; 2FA codes silently discarded between requests	Added AddSession() and UseSession() in correct pipeline order in Program.cs	TC-03, TC-15

The sequential structure of the testing program is what kept the diagnostic effort manageable. Because integration tests ran first, the hashing inconsistency and column mapping errors were still localized to specific modules when they appeared each had a single, identifiable cause. By the time system tests ran, those causes had already been addressed, so the system tests could ask a cleaner question: do all the workflows hold together? Acceptance tests then confirmed operational utility that neither of the earlier levels could assess. Summers (2020) describes staged verification as what distinguishes controlled development from ad hoc integration, and the defect record from this project provides concrete evidence for that distinction.

4.3 Performance Metrics

Evaluation was structured around two complementary evidence types timing measurements and usability observation because each answers a question the other cannot. Timing confirms whether the system performs within acceptable ranges; usability observation confirms whether it is comprehensible and navigable for its intended users. Neither is a substitute for the other, and combining them produces a more complete picture of system quality than either alone (LaunchNotes, 2023). Table 2 presents the complete results.

Table 2. System Performance Evaluation Results

Metric Type	Indicator	Result	Interpretation
Quantitative	Login Response Time	1.2 seconds	Acceptable for local prototype; establishes performance baseline
Quantitative	CSV Export Generation	2.4 seconds	Efficient for current dataset; scales with optimization
Quantitative	Build Validation	Zero errors	Confirms stable compilation and module integration
Qualitative	Interface Usability	Positive	Consistent navigation; clear action patterns across modules
Qualitative	Workflow	All flows	All primary workflows confirmed end-to-end

Metric Type	Indicator	Result	Interpretation
	Completeness	pass	
Qualitative	Notification Delivery	Successful	TechOps emails received and readable in all test runs

Login averaged 1.2 seconds, covering the round-trip from form submission through credential hashing, database validation, session establishment, and dashboard render. CSV export averaged 2.4 seconds the longer time reflects the additional steps of querying the schedule table, iterating rows with StringBuilder, encoding the output as UTF-8, and constructing the file response. The 2:1 difference is structurally expected given the difference in workload, and neither figure indicates a performance problem at this stage. The build output was clean across every run: zero compilation errors, zero warnings, every project reference resolved. Response time is a standard benchmark for web application latency evaluation (GeeksforGeeks, 2025), and both measured values sit within the range that would be expected for a single-user local prototype with no caching layer.

The qualitative review focused on three aspects: whether the navigation was intuitive, whether workflows required unnecessary steps, and whether the interface communicated status clearly. The fixed horizontal navigation bar with clearly labelled module links gives users a consistent orientation point on every screen. The action button pattern View, Complete, cancel appears in the same position on both the Schedules and Shell Requests list screens, reducing cognitive load when switching between modules. Dashboard summary cards deliver operational context immediately without requiring a user to drill into individual records.

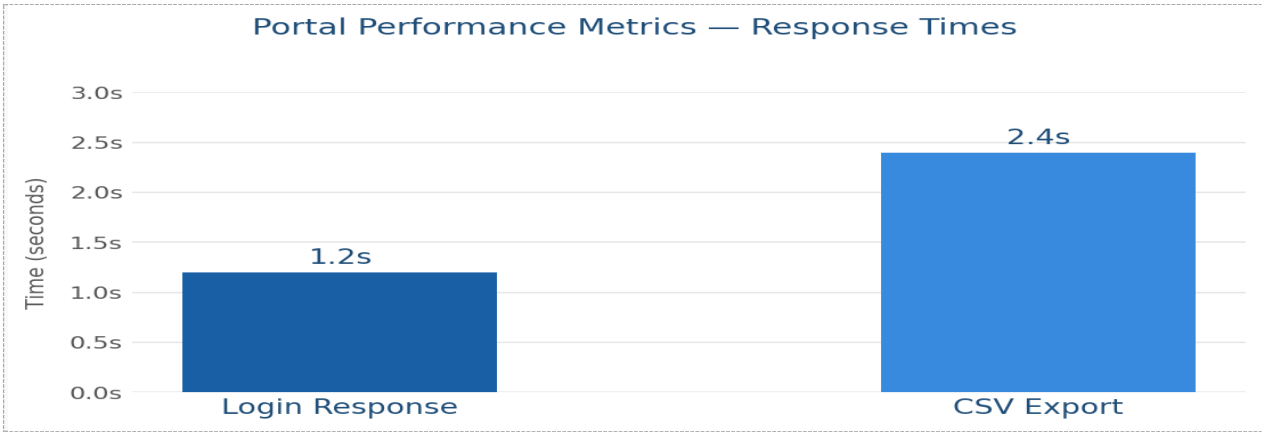


Figure 10. Column chart comparing Login Response time (1.2 s) and CSV Export time (2.4 s)

Chapter 5: Discussion

5.1 Interpretation of Results in Context of Literature

Every major technical decision in this project had a specific rationale drawn from the literature, and the outcomes of those decisions are traceable back to the reasoning that motivated them.

Running three testing levels in sequence rather than applying one comprehensive review produced findings that complemented rather than duplicated each other. Integration tests exposed the hashing mismatch and column mapping problems while their causes were still isolated to specific modules, making diagnosis and repair straightforward. System tests ran against a codebase from which those defects had already been removed, which meant they could focus on workflow completeness rather than fundamental correctness. Acceptance tests answered questions that code-level testing cannot: whether a real email arrived, and whether the CSV rendered correctly in a spreadsheet application. Summers (2020) argues that verification at each stage is what separates controlled development from ad hoc integration, and the testing sequence demonstrated that argument concretely.

The performance measurements establish a baseline, not a ceiling. Any latency figure needs its context to mean anything this one means a single user, a development laptop, no caching layer, and a SQL Server instance sharing resources with the rest of the development environment (Wells, 2025). Under those conditions, 1.2 seconds for login and 2.4 seconds for a full CSV export are reasonable results. The 2:1 ratio reflects the additional processing in the export path and is structurally expected. Neither figure is a signal that the implementation needs to change before the next development phase begins.

The prototype model's practical value showed up most clearly in how integration failures behaved: they were small and localized rather than large and systemic. Because each unit extended working, tested code from the previous one, no integration event involved combining a large batch of unverified changes. When something failed, the cause was identifiable within the changes made in that unit rather than somewhere in weeks of accumulated work. This is the concrete benefit Laptick (2022) attributes to iterative prototype development, and the defect history across Units 5 through 7 demonstrates it rather than just asserting it.

The Unit 4 CI/CD reflection identified GitHub Actions as the appropriate pipeline tool for this repository. The defect history from Units 5 through 7 gave that analysis a concrete illustration: the EF Core column mapping errors would have produced build failures seconds after being committed, had a dotnet build step been active. Instead, they appeared during manual testing sessions, after the developer's mental context had shifted away from the specific change that introduced them. The additional diagnostic time was the direct cost of the missing pipeline. Shortening that feedback gap is precisely the quality benefit Narasimhan et al. (2025) attribute to CI/CD automation, and experiencing the alternative confirmed rather than complicated that argument.

5.2 Limitations

Five limitations shape the scope of the current results.

First, every timing measurement was made with one user on one machine. The 1.2-second login and 2.4-second export are single-user, local-hardware figures. Whether those times hold under ten simultaneous users, or under a schedule table with ten times the current row count, is

genuinely unknown. Any future decision about hosting configuration should begin with load testing that answers that question with data rather than assumption.

Second, all testing was manual. The three-level structure was disciplined, but manual execution means every code change requires a developer to manually repeat affected test sequences. AWS (n.d.) identifies this as the fundamental scaling problem with manual testing: the cost of regression coverage grows with the code base, and the temptation to skip tests under time pressure grows with it.

Third, role enforcement currently stops at the view layer. Navigation links are shown or hidden based on the logged-in user's role, but the controller actions behind those links carry no [Authorize(Roles)] attribute. Someone who knows the URL of an admin-only action can call it directly, bypassing the navigation-level gate entirely. This gap is acceptable during the prototype phase where the application runs on a developer's laptop but ceases to be acceptable the moment any user outside the development team is given credentials.

Fourth, the SendGrid dependency cannot be mocked for testing. Every test case that exercises an email-sending path makes a real API call, requiring network access, valid credentials, and API quota. Replacing the live service with a configurable mock for development and test environments would decouple the test suite from the external dependency and make tests faster and runnable offline.

Fifth, no requirements traceability matrix links the ten functional requirements in Appendix D to the test cases in Table 3. Producing that matrix would make it immediately visible which requirements are covered by testing and which are not and would highlight coverage gaps when new features are added.

5.3 Maintenance Plan

The maintenance plan organizes ongoing work into four categories. Corrective maintenance handles defects: reproduce on the development branch, fix, re-run the affected test cases, merge to main with a patch version increment. Adaptive maintenance tracks the planned feature roadmap — Jira integration at v1.1.0 (next minor release), advanced analytics at v1.2.0 — and accommodates external dependency changes that require application updates. Perfective maintenance improves quality without breaking existing behaviour: adding [Authorize(Roles)] to controllers, applying EF Core AsNoTracking() on read-only query paths, and building dashboard trend charts. Preventive maintenance keeps the system healthy: monthly NuGet security updates, weekly Serilog log reviews, database backup verification, and periodic checks that no credentials have appeared in the commit history.

Bugs and enhancement requests both flow through GitHub Issues, each with required fields that make the history readable and the work traceable. A bug record must include reproduction steps, expected versus actual behaviour, a severity classification, and the version where the issue was first observed. An enhancement request carries an enhancement label and moves through four defined stages submitted, triaged at the monthly backlog review, scheduled to a milestone, and delivered with a version increment so that nothing is silently deferred or forgotten. The value of this structure is that any future developer or reviewer can reconstruct the decision history of the project from the Issues log alone. That traceability is what Bennett (2024) identifies as the defining property of software configuration management: controlled, auditable change rather than ad hoc intervention.

Table 6. Proposed Maintenance Schedule

Frequency	Type	Activity	Deliverable
Weekly	Corrective / Preventive	Review Serilog log files for errors or anomalies	GitHub Issue opened for any finding
Weekly	Preventive	Verify database backup integrity and restore test	Backup log reviewed and confirmed
Weekly	Preventive	Check SendGrid delivery reports for bounced or failed emails	Report reviewed; issues escalated if found
Monthly	Preventive	Apply NuGet and .NET SDK security updates; confirm dotnet build passes	Packages updated; clean build confirmed
Monthly	Adaptive	Triage GitHub Issues; groom enhancement request backlog	Backlog updated; items assigned to milestone
Quarterly	Perfective	Re-run login and export performance measurements; compare to baseline	Results documented; optimization tasks created if needed
As needed	Corrective	Bug reported: reproduce on dev → fix → test → merge → patch release	Fix deployed; GitHub Issue closed

Table 5. Post-Deployment Risk Register

ID	Severity	Risk	Mitigation	Contingency
R-01	High	No server-side [Authorize] on controllers; URL access bypasses role gate	Add [Authorize(Roles)] to all restricted actions	Revoke sessions; deploy auth patch on dev branch immediately
R-02	Medium	API credentials accidentally committed to repository	Store secrets as environment variables; add .gitignore rules; enable secret scanning	Rotate credential; purge with git-filter-repo; audit commit history

ID	Severity	Risk	Mitigation	Contingency
R-03	Medium	EF Core migration drift when schema changes are not applied consistently	Document and run dotnet ef database update in setup sequence	Generate corrective migration; test on dev database before applying
R-04	Medium	SendGrid API outage causing notification delivery failure	Log all dispatch attempts; monitor SendGrid status	Queue failed details locally; replay on restoration
R-05	Low	Navigation links visible to unauthorized roles (UI-layer only check)	Hide inaccessible links via role-based Razor conditionals	Triage as corrective ticket; fix on dev branch; patch release

The most urgent risk is R-01: the absence of server-side role enforcement. Adding [Authorize(Roles)] attributes to the appropriate controller actions is a targeted, low-risk code change that closes the gap without restructuring anything else. It is the precondition for sharing the portal with any user outside the development team. The risk register will be reviewed quarterly, with new risks added and severity reclassified as mitigations are applied.

5.4 Recommendations for Future Work

Five improvements are recommended for the next development cycle, ranked by urgency.

The highest-priority improvement is adding [Authorize(Roles)] attributes to all restricted controller actions. This closes the access control gap identified in the limitations and makes authorization decisions enforceable at the HTTP layer regardless of how a request is constructed. It is the change that must happen before the portal is shared with any user outside the development team.

The second improvement is an automated xUnit test project covering authentication, user management, and CSV export. The test scenarios are already defined in Table 3 converting those rows into automated test methods is a translation task rather than a design one, because the inputs, preconditions, and expected outputs are already documented (Rana, 2023). What changes is the cost of regression coverage: instead of manually repeating test sequences after every code change, the test runner confirms correctness automatically on every commit.

The third improvement is the GitHub Actions pipeline: a workflow file that triggers on development branch pushes, runs dotnet restore, dotnet build, and dotnet test in sequence, and blocks any merge to main if a step fails. Every commit becomes self-validating, and the merge gate becomes mechanical rather than manual. Red Hat (2025) describes this shift from treating deployment as a periodic high-risk event to treating build verification as a routine property of daily engineering work as the defining characteristic of mature CI/CD adoption.

The fourth improvement is completing the Jira integration module and the advanced analytics dashboard. Connecting shell request records to Jira tickets eliminates the duplicate data entry that currently exists between the portal and TechOps's daily ticketing workflow. Adding trend charts to the dashboard upgrade completions over time, shell request volume by month replaces manually assembled CSV analysis with information available immediately.

The fifth improvement is conducting structured load testing with Apache JMeter or k6. Even a scenario simulating ten simultaneous login sessions would replace the current assumption that single-user performance generalizes to multi-user conditions with actual evidence, and would provide specific inputs to any future SQL Server query optimization and hosting configuration decisions.

Chapter 6: Conclusion and Future Work

6.1 Summary of Findings and Contributions

The Upgrade Portal project delivered a working prototype that addresses a specific, real operational problem. Healthcare IT teams that track EMR upgrade schedules in spreadsheets and submit environment requests through general-purpose ticketing tools face predictable coordination failures. The portal replaces those fragmented workflows with a single, purpose-built application that enforces request completeness, automates notifications, and makes status visible to every authorized user in real time.

Eight development units progressed from a problem statement through architecture design, iterative implementation, structured testing, quantified evaluation, and maintenance planning. All ten documented test cases across integration, system, and acceptance levels passed. Three defects a SHA-256 hashing inconsistency, an EF Core column mapping error, and a missing session middleware registration were identified and resolved before any change reached the main branch. The portal is published at https://rauf600.github.io/UpgradePortal with source code available at https://github.com/Rauf600/UpgradePortal. Performance evaluation recorded a 1.2-second login response and a 2.4-second CSV export with zero build errors and a consistently positive qualitative usability assessment.

The practical contribution is a demonstrated solution to a workflow problem that affects real IT operations teams. When a support agent submits a shell request through the portal, TechOps receives a complete, structured notification without any manual forwarding step. When an administrator needs to review the upgrade schedule, every record is visible with its current status

and history. The reduction in coordination overhead has measurable downstream implications for the timeliness of environment provisioning and client readiness.

The software engineering contribution is a demonstration that professional discipline does not require a large team or enterprise resources. MVC architecture, security-first design, semantic versioning, structured testing, and configuration management were all applied consistently throughout the project not described in planning documents and then set aside but embedded in actual implementation decisions. Summers (2020) argues that this is what distinguishes software engineering practice from software development: the practices are not add-ons applied at the end but habits that shape every technical choice. The portal's codebase, version history, test case documentation, and maintenance plan provide the evidence for that claim rather than just restating it.

6.2 Lessons Learned

Three lessons from this project carry the most practical weight for similar development contexts.

The first is that shared logic must live in exactly one place. The hashing inconsistency that became Defect D-01 was created the moment the same algorithm was written twice in different modules. Any computation that needs to produce the same output everywhere it is called belongs in a single utility class, called from every site. Writing it twice is not a style problem it is a defect waiting to surface at the worst possible moment.

The second is specific to ASP.NET Core but generalizes broadly: explicit registration is required for every capability, and missing registration produces silent failure rather than an error. The 2FA session defect produced no build error, no start-up warning, and no runtime exception just verification codes that were silently discarded. Two lines in Program.cs resolved the issue.

Reviewing the middleware configuration before implementing any feature that depends on request-scoped state would have prevented that sequence entirely.

The third lesson is about documentation as a quality mechanism rather than a reporting requirement. Each unit produced more structured test documentation than the one before it, and the quality of each unit's work reflected that progression. The EF Core mapping errors would have been caught earlier if a structured test table had been part of Unit 4's deliverable. Developer confidence that something works and documented evidence that it works are not the same thing, and the gap between them is where defects hide.

6.3 Challenges Encountered

Three categories of challenge shaped the development experience. Technical complexity concentrated at integration boundaries: the controller-to-EF Core handoff, the EF Core-to-SQL Server connection, and the service layer-to-SendGrid API call each introduced independent failure modes. A misconfigured await in one place could produce a symptom that looked like a problem somewhere else entirely. The habit of confirming each integration point worked correctly before combining it with the next proved to be the most reliable diagnostic approach.

Scope pressure was persistent rather than dramatic. The Unit 4 presentation described a more complete system than the following three units could deliver without cutting corners on quality. The decision to defer the Settings module and Jira integration in favour of a thoroughly tested core was the right call, but it required making that call explicitly at each unit boundary rather than assuming the earlier decision still held.

Manual testing at this scale was a time problem. Running documented test sequences across five modules after every code change consumed a disproportionate share of each unit's development

time. The experience made the recommendation for an automated xUnit test project feel less like a theoretical improvement and more like a necessary one.

6.4 Future Development Roadmap

The five improvements described in Section 5.4 define the next development cycle in priority order. Beyond those near-term items, two longer-horizon enhancements would significantly expand the portal's operational value. The first is a notification preference system that allows administrators to configure which events trigger which email templates, making the notification framework more flexible for organizations with different TechOps routing requirements. The second is a customer self-service status view a read-only interface allowing clients to check their scheduled upgrade date and status without requiring an internal account, which would reduce the volume of status inquiry calls between clients and the support team.

The portal has a solid foundation for continued development. Every module is implemented, documented, and tested. The architecture supports planned enhancements without structural changes. The version control history provides a complete record of every design decision, implementation choice, and defect resolution. With the improvements described above, the portal is well-positioned to transition from a prototype that demonstrates the concept to a tool that IT teams can genuinely rely on in daily operations.

Chapter 7: Digital Portfolio

7.1 Portfolio Strategy and Best Practices

A portfolio occupies a different position in the professional communication landscape than a résumé or a technical report. A résumé describes what was done; a report explains how and why; a portfolio shows the work itself. For a software engineering capstone, that means giving a technical reviewer a direct path from a visual overview to the actual code, the actual tests, and the actual documentation — without friction, and without requiring them to ask for access. The quality of that path is what Carnegie Mellon University School of Computer Science (2024) identifies as the distinguishing property of a strong developer portfolio: technical depth that is genuinely accessible, not buried under navigation or scattered across platforms.

The Upgrade Portal portfolio is distributed across two platforms serving different reviewer needs. GitHub Pages at https://rauf600.github.io/UpgradePortal provides the visual entry point: a landing page with a system overview, module screenshots, architecture and ERD diagrams, and a link to the demonstration video. The project repository at https://github.com/Rauf600/UpgradePortal provides the technical depth: source code, commit history, release tags, and the structured README. A LinkedIn entry is planned and will link to both once published. LinkedIn's Featured section is one of the highest visibility surfaces available for showcasing professional work, placing selected projects directly on the profile where they are seen by anyone who views it (LinkedIn, 2023).

7.2 Deliverables Included in the Portfolio

The portfolio includes deliverables spanning the full development lifecycle. The GitHub Pages site presents prototype screenshots of all six modules login, dashboard, schedule management, shell requests, user management, and CSV export each captioned and organized in the same sequence as the capstone report chapters. Architecture and ERD diagrams are embedded directly in the site for immediate visual context. The capstone final report is linked as a downloadable PDF, giving any reviewer who wants full technical depth direct access to the complete documentation.

The GitHub repository contains the full application source code organized in the standard ASP.NET Core project structure, with a structured README providing a project overview, prerequisite list, setup commands, environment variable documentation, and links to the GitHub Pages site and the five release tags. Release notes for v1.0.0 through v1.0.4 document what changed in each patch, what was tested, and what known issues remain. Demonstration videos show complete end-to-end workflows login through dashboard, schedule creation, shell request submission with email notification, and CSV export without narration gaps or unexplained jumps. Guiding visitors from overview to detail rather than presenting everything at once is a principle W3Schools (n.d.) identifies as central to effective portfolio design. The eight-section structure of the portal's GitHub Pages site applies that principle directly: a reviewer who has five minutes sees the problem and solution first; a reviewer who wants technical depth finds it in the later sections without searching.

7.3 Organization and Formatting

The GitHub Pages site is organized into eight sections that guide a reviewer through the project in the same sequence it was built: Project Overview, Features and Functionality, Architecture and Design, Testing and Evaluation, Deployment and Maintenance, Source Code, Documentation, and Demo Videos. This progression means a reviewer who only has five minutes sees the problem, the solution, and the key evidence first; a reviewer who wants technical depth finds it in the later sections without having to search.

All screenshots use consistent formatting and captions. Every figure on the site matches the caption style used in the capstone report, creating a coherent cross-reference. The README is written to be readable by someone with no prior knowledge of the project it provides enough context to understand what was built and enough instruction to run it locally. Configuring GitHub Pages to serve from the repository's docs/ folder keeps the portfolio and the source code in the same repository, so a reviewer can move between the visual presentation and the underlying code without switching platforms or asking for access. HoffsTech (2023) demonstrates that this setup takes minutes to configure and requires no separate deployment infrastructure a practical consideration that kept portfolio publication from becoming a distraction from the capstone work itself.

7.4 Ensuring the Portfolio is Polished and Accessible

Several specific steps were taken to ensure the portfolio signals professional care to any reviewer. Sensitive configuration values the SendGrid API key, the database connection string, and the TechOps notification email address are removed from the public repository and documented only as environment variable names in the README. Any reviewer who wants to

run the portal locally receives clear instructions for supplying their own credentials. The same configuration discipline applied throughout the project's development credentials in environment variables, never in source code applies to the public repository as well. A reviewer who clones the repository receives the application structure and clear documentation of what credentials to supply; they do not receive the actual secrets. Bennett (2024) describes this separation as the foundation of auditable software delivery, and the public portfolio is the natural continuation of that practice.

The demonstration videos were recorded at a resolution sufficient for all module text to be readable without zooming, and each video shows a complete workflow from start to finish rather than isolated screenshots. Screenshots are exported at sufficient resolution for the capstone report's print layout. The repository commit history is clean each commit message describes what changed and why, which gives any technical reviewer reading the history a narrative of the development process rather than a sequence of undifferentiated changes. A GitHub repository with meaningful commit messages and a well-structured README communicates professional engineering discipline in a way that a list of skills on a résumé cannot. Every commit message in this repository describes what changed and why, which means any reviewer reading the history can reconstruct the development narrative. GlobalTechFix (2024) identifies this kind of repository organization as one of the clearest signals a developer can send to a technical interviewer about their engineering habits.

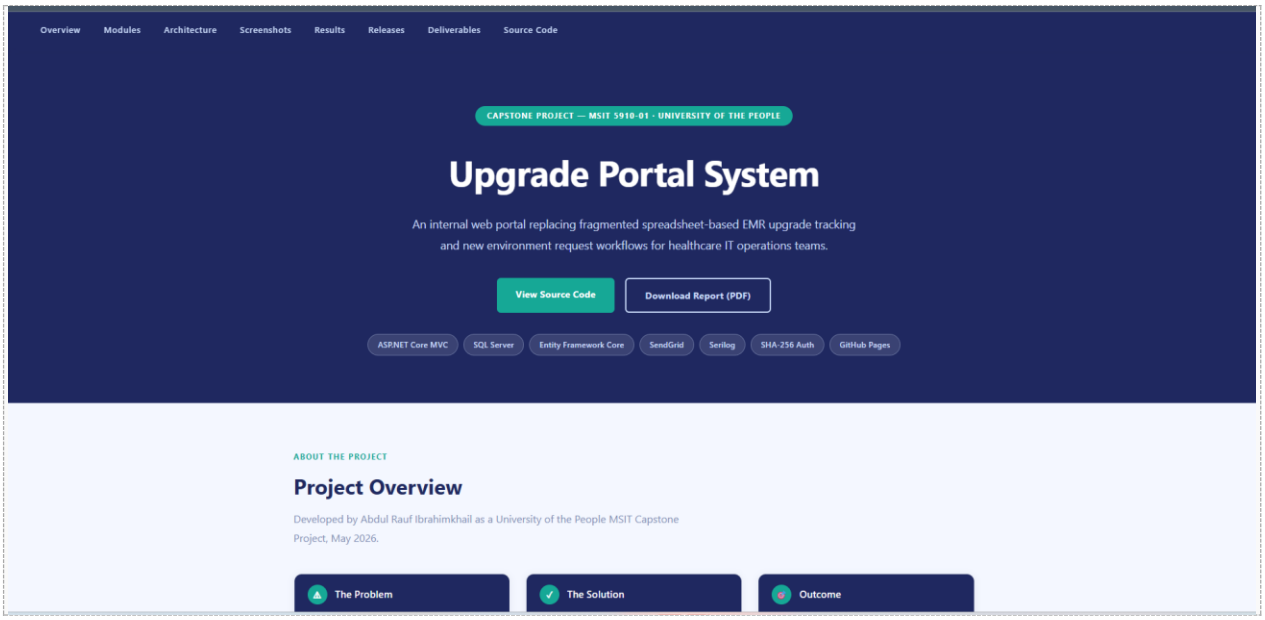


Figure 15. GitHub Pages portfolio landing page showing project overview, module screenshots, and navigation

References

Alex, H. (2023). 5 types of testing software every developer needs to know! [Video]. YouTube.

Alsubaei, F., Abuhussein, A., & Shiva, S. (2019). Security and privacy in the Internet of Medical Things: Taxonomy and risk assessment. IEEE Access, 7, 489–507. https://doi.org/10.1109/ACCESS.2018.2886416

Amazon Web Services. (n.d.). What is unit testing? AWS. https://aws.amazon.com/what-is/unit-testing/

Bennett, L. (2024, August 13). Software configuration management in software engineering. Guru99. https://www.guru99.com/software-configuration-management-tutorial.html

BrowserStack. (2025). What is system testing? BrowserStack. https://www.browserstack.com/guide/system-testing

ByteByteGo. (2023, June 6). Top 5 most-used deployment strategies [Video]. YouTube. https://www.youtube.com/watch?v=AWVTKBUnoIg

Carnegie Mellon University School of Computer Science. (2024, December 23). How to build a digital profile as a software developer. CMU TechBridge Coding Bootcamp Blog. https://bootcamps.cs.cmu.edu/blog/build-digital-profile-software-developer

Davies, S. (2024). Blueprints: Creating, describing, and implementing designs for larger-scale software projects (Version 2.5). University of Mary Washington. https://open.umn.edu/opentextbooks/textbooks/blueprints-creating-describing-and-implementing-designs-for-larger-scale-software-projects-davis

GeeksforGeeks. (2025). Measuring software quality using quality metrics. https://www.geeksforgeeks.org/software-engineering/measuring-software-quality-using-quality-metrics/

GeeksforGeeks. (2025). System testing — Software testing. https://www.geeksforgeeks.org/software-testing/system-testing/

GlobalTechFix. (2024, June 29). How to upload project on GitHub [Video]. YouTube.

HoffsTech. (2023, March 17). How to create a GitHub portfolio in 5 minutes using GitHub Pages [Video]. YouTube.

Hrupa, A. (2024). What is system testing in software testing. Testfort. https://testfort.com/blog/what-is-system-testing

Kruse, C. S., Frederick, B., Jacobson, T., & Monticone, D. (2017). Cybersecurity in healthcare: A systematic review of modern threats and trends. Technology and Health Care, 25(1), 1–10. https://doi.org/10.3233/THC-161263

LaunchNotes. (2023, September 1). Qualitative vs quantitative metrics: A comprehensive comparison. LaunchNotes. https://www.launchnotes.com/blog/qualitative-vs-quantitative-metrics-a-comprehensive-comparison

Laptick, S. (2022, November 9). How SDLC prototype model helps build comprehensive apps when requirements are unclear. XB Software. https://xbsoftware.com/blog/prototype-model-sdlc/

Letaw, L. (2024). Handbook of software engineering methods. Oregon State University. https://open.oregonstate.education/setextbook/

LinkedIn. (2023, March 14). LinkedIn Portfolio: A step-by-step guide to showcase your work. LinkedIn Pulse. https://www.linkedin.com/pulse/linkedin-portfolio-step-by-step-guide-showcase-your-work/

Narasimhan, A., Sham, R., & Subramanian, H. (2025, October 15). A beginner's handbook to DevOps [White paper]. Dell Technologies. https://awsprod.merlot.org/merlot/viewMaterial.htm?id=773474552

Oguz, A. (2025). Project management (2nd ed.). MSL Academic Endeavors. https://pressbooks.ulib.csuohio.edu/projectmanagement2ndedition/

Parzych, D. (2019, December 30). 8 must-read DevOps articles for success in 2020. Opensource.com. https://www.opensource.com/article/19/12/devops-resources

Project Management Institute. (2021). A guide to the project management body of knowledge (PMBOK Guide) (7th ed.). https://www.pmi.org

Rana, K. (2023, September 23). What is a test case? Test case examples. ArtOfTesting. https://artoftesting.com/test-case

Red Hat. (2025, April 29). What is DevOps? Red Hat. https://www.redhat.com/en/topics/devops/what-is-devops

Summers, B. L. (2020). Effective methods for software engineering. Auerbach Publishers.

Sumo Logic. (n.d.). What is software deployment? Sumo Logic. https://www.sumologic.com/glossary/software-development/

Thales. (n.d.). What is a software maintenance process? Thales Group. https://www.thalesgroup.com/en/markets/digital-identity-and-security/banking-payment/issuance/id-verification/glossary/software-maintenance

Walker, V. (2022, March 16). 14 software architecture design patterns to know. Red Hat. https://www.redhat.com/en/blog/14-software-architecture-patterns

Wells, J. (2025, April 16). Top 12 software quality metrics to measure and why. LinearB. https://linearb.io/blog/software-quality-metrics

W3Schools. (n.d.). How to create a portfolio. W3Schools. https://www.w3schools.com/howto/howto_website_create_portfolio.asp

Appendices

Appendix A: System Prerequisites and Setup Commands

The following prerequisites must be installed on the development machine before running the portal:

- Windows 10/11 or Windows Server
- ASP.NET Core Runtime (.NET 8 or later)
- .NET SDK 8.0 or later
- SQL Server Express or Developer edition
- Visual Studio 2022 or Visual Studio Code
- SendGrid account with active API key
- Modern web browser (Chrome, Edge, or Firefox)

Setup command sequence:

```
git clone https://github.com/Rauf600/UpgradePortal.git
cd UpgradePortal
dotnet restore
dotnet build
dotnet ef database update
dotnet run
```

Appendix B: Required Environment Variables

`ConnectionStrings:DefaultConnection` — SQL Server connection string for the portal database

`SendGrid:ApiKey` — SendGrid Web API authentication key

`SendGrid:FromEmail` — Sender email address for portal notifications

`SendGrid:FromName` — Display name for outbound emails

`SendGrid:TechOpsEmail` — TechOps team recipient address for shell request alerts

`ASPNETCORE_ENVIRONMENT` — Runtime environment flag — set to Development for local use

Appendix C: Repository Links

Main branch (stable releases): https://github.com/Rauf600/UpgradePortal/tree/main

Development branch (active work): https://github.com/Rauf600/UpgradePortal/tree/development

GitHub Pages portfolio: https://rauf600.github.io/UpgradePortal

Appendix D: Functional Requirements

FR-01 — The system shall allow authorized users to submit EMR upgrade requests through a structured form with mandatory field validation.

FR-02 — The system shall allow authorized users to submit New Shell Request forms for new client sites or environments.

FR-03 — The system shall validate all mandatory fields server-side before a request can be saved to the database.

FR-04 — The system shall provide role-based dashboards showing request status and summary metrics.

FR-05 — The system shall allow TechOps users to review, update the status of, and complete requests.

FR-06 — The system shall maintain a history log of status changes and significant user actions through Serilog.

FR-07 — The system shall generate operational reports and downloadable CSV exports for management review.

FR-08 — The system shall send automated email notifications to TechOps on shell request submission via SendGrid.

FR-09 — The system shall support two-factor authentication for all user accounts where the 2FA flag is enabled.

FR-10 — The system shall enforce role-based access control across all modules and restricted controller actions.

Appendix E: Maintenance Schedule (Full)

Refer to Table 6 in Chapter 5 for the complete maintenance schedule. The schedule is designed for the current single-developer prototype context and will be expanded with team assignments and automated CI/CD triggers when additional contributors join the project or when the GitHub Actions pipeline is implemented.